

Recap — Assignment #5: The inventory panel

Recap — Assignment #5: The inventory panel

Branch: `feature/inventory-panel` **Trello cards:** Person B, Sprint 2 — *Build inventory panel, Build item cards, Add item quantities, Add invalid-item states*

Date: 2026-08-19

Assignment #4 taught the **server** to pick up and use items. This assignment is the other half: it lets the **player see** any of it. It is your first assignment that is pure frontend since #3.

Part 0 — What we built

Before this, the game screen had a box in the right-hand column with one hard-coded sentence in it:

```
<section className="game-panel" aria-label="Inventory">
  <h2>Inventory</h2>
  <p>Collected items will appear here.</p>
</section>
```

That sentence never changed. It was a placeholder left behind in assignment #3.

Now the box lists what the player is actually carrying, with a **Use** button on each item. A second list appeared in the middle of the screen — **Items here** — showing what is lying on the floor of the room you are standing in, with a **Pick up** button.

Part 1 — The one idea that shapes everything

The data you need is split across two places, and you have to join it yourself.

What the player carries is stored as small as possible, on the game session:

```
game.inventory = [{ itemId: "city_map", quantity: 1 }]
```

That is an id and a number. There is no name in it, no description, no type.

Everything readable lives somewhere else entirely — on the scenario:

```
scenario.items = [
  {
    id: "city_map",
    name: "City Map",
    description: "A traders' map of Pompeii showing the roads out
      toward the harbor.",
    type: "tool",
    locationId: "forum",
    effect: { type: "none", amount: 0 },
  },
]
```

To draw one row on screen you need both halves. The inventory tells you **which item and how many**. The scenario tells you **what it is**. You join them by matching `entry.itemId` against `item.id`.

Why is it stored this way? Because the scenario is one shared document that every player reads, and the game session is private to one player. If the inventory copied the name and description into itself, then fixing a typo in an item description would only fix it for new games — every game already saved would keep the old text forever. Storing the id alone means the description is looked up fresh every time.

This is called **normalisation**: store each fact once, refer to it by id everywhere else. You will meet the same shape again in the Mission panel, which stores `{ objectiveId, status }` and looks the title up on the scenario.

Part 2 — One card component, two lists

`ItemCard` is used by **both** lists. The same card appears in the inventory with a **Use** button, and on the floor with a **Pick up** button.

That works because the card does not know what its button does. The button's text and behaviour are handed in from outside:

```
<ItemCard item={item} actionLabel="Pick up" onAction=
  {handlePickUpItem} />
<ItemCard item={item} quantity={2} actionLabel="Use" onAction=
  {handleUseItem} />
```

Prop — an input handed to a component, like an argument to a function.

`actionLabel` and `onAction` are props.

`onAction` is a prop holding a **function**. The card calls it when the button is clicked, and the page decides what actually happens. This pattern has a name: the card **raises an event** and the parent **handles** it. It is what keeps the card reusable — the card knows how to look, the page knows what to do.

The alternative was writing two nearly identical card components and keeping their styling in sync by hand forever. Passing two props is cheaper.

Part 3 — Quantities

The quantity badge only renders when a quantity is actually passed:

```
{quantity === undefined ? null : <span className="item-  
card_quantity">&times;{quantity}</span>}
```

Note it tests `=== undefined`, not just truthiness. Writing `{quantity && ...}` would be a bug: if `quantity` were `0`, JavaScript treats `0` as false, and React would print a literal `0` on the screen instead of hiding the badge. That is one of the most common React bugs there is.

Items on the floor pass no quantity at all, so they show no badge. Carried items always show one, even at `×1`, so the number is always visible where it matters.

Today every quantity is `1`, because `pickUpItem` always pushes `{ itemId, quantity: 1 }` and refuses a second copy. The badge is ready for the day something stacks.

Part 4 — The invalid-item states, and the trap in them

Assignment #4 defined six error codes. Each one now has a sentence written for the player, in `client/src/utils/itemErrors.js`:

Server code	HTTP	Shown to the player
<code>ITEM_NOT_FOUND</code>	404	That item does not exist in this scenario.
<code>ITEM_NOT_HERE</code>	409	That item is not in this location.
<code>ALREADY_HAVE_ITEM</code>	409	You are already carrying that item.
<code>ITEM_NOT_IN_INVENTORY</code>	409	You are not carrying that item.
<code>ITEM_NOT_USABLE</code>	409	That item has no use.
<code>VALIDATION_ERROR</code>	400	That action did not name an item.

The message appears **on the card you clicked**, in red, not in some corner of the screen. The card gets a red border too.

The trap

The obvious way to decide "is this an item error?" is to look at the error code. That is wrong, and it took a moment to spot.

`VALIDATION_ERROR` is **not** item-only. `move` throws exactly the same code when a destination is missing — `server/services/gameActionService.js:10`. So sorting errors by code would sometimes print a failed **movement** error inside the inventory box.

The fix is to remember which **action** failed, not just which code came back. The store grew one field:

```
error = null;
failedAction = null; // the action object that produced `error`
```

It is set in the same place the error is set:

```
} catch (error) {
  runInAction(() => {
    this.error = error;
    this.failedAction = action;
    this.actionPending = false;
  });
}
```

Now the page can ask a precise question — *did a `USE_ITEM` fail, and on which item?* — instead of guessing from a code that two different features share.

Lesson worth keeping: an error code answers *what went wrong*. It does not answer *what the user was trying to do*. If your UI needs the second question answered, you have to record it yourself.

A bug caught before it ran

The first wiring passed the same error to both lists. A failed **Use** on the City Map would have shown twice — once in the inventory, and once under *Items here*, where you never clicked. Both lists received `itemError`, and neither had any way to tell whose button it was.

Split into two, so each list only ever shows errors from its own button:

```
const pickUpError = failedActionType === "PICK_UP_ITEM" ? itemError :
  null;
const useError = failedActionType === "USE_ITEM" ? itemError :
  null;
```

Part 5 — Small defensive choices

An item that vanished from the scenario. If a saved game carries an id that is no longer in `scenario.items` — someone edited the scenario after the game was saved — the panel shows the row anyway, labelled with the raw id and "This item is no longer part of the scenario." The alternative was `.find()` returning `undefined` and the page crashing on `item.name`. Never let a lookup miss take the screen down.

The Use button stays enabled on unusable items. The City Map cannot be used. The button could have been greyed out. It was left clickable on purpose, so the server stays the one deciding what is usable, and so `ITEM_NOT_USABLE` is actually reachable — showing it was one of the four cards. Easy to reverse if you would rather grey it out.

Empty states. An empty inventory says "You are not carrying anything." rather than leaving a blank box. An empty floor renders nothing at all, so rooms with no items stay clean.

Part 6 — The button nobody owned

Here is something worth noticing about how the board is split.

There is a Trello card for the `PICK_UP_ITEM` **action** (done in #4), and cards for the inventory **panel**. There is no card anywhere for a **pick-up button**. Neither Person B nor Person C had it.

Without it the inventory can never fill, and the panel is untestable in the real app. So `LocationItems.jsx` was added to close the gap — about 45 lines, reusing `ItemCard`.

Lesson: a board split by layer — "server action" to one person, "UI panel" to another — can leave the join between them unassigned. Nobody notices until someone tries to use the feature end to end.

Part 7 — Words to remember

Word	What it means
Component	A reusable piece of screen; a function that returns markup
Prop	An input handed to a component, like a function argument
Normalisation	Store each fact once, refer to it by id elsewhere
Lifting state up	The child raises an event, the parent decides what happens
Empty state	What the screen shows when a list has nothing in it
Defensive rendering	Assuming a lookup can miss, and not crashing when it does

Part 8 — Files

New — 5 files

File	What it does
<code>client/src/components/game/ItemCard.jsx</code>	One item: name, type badge, description, quantity, one button
<code>client/src/components/game/ItemCard.css</code>	Its styling, including the red invalid state
<code>client/src/components/game/InventoryPanel.jsx</code>	The carried-items list, with Use
<code>client/src/components/game/InventoryPanel.css</code>	Its styling
<code>client/src/components/game/LocationItems.jsx</code>	Items on the floor here, with Pick up
<code>client/src/components/game/LocationItems.css</code>	Its styling
<code>client/src/utils/itemErrors.js</code>	The six codes turned into player sentences

Changed — 2 files

File	What changed
<code>client/src/stores/gameStore.js</code>	Added <code>failedAction</code> , cleared in <code>loadGame</code> and at the start of <code>runAction</code> , set in the <code>catch</code>
<code>client/src/pages/games/GamePage.jsx</code>	Placeholder replaced; two handlers added; errors routed to the right list

`client/src/utils/` is a new folder. It is the first one in this client.

Part 9 — Merging with Person C

This branch was cut **after** merging Person C's 15 commits, not before. That mattered: they had rewritten `GamePage.jsx` in the meantime — filling the Mission box with a real `MissionPanel`, and wrapping the whole board in a status check so a finished game shows a Victory or Game-Over screen instead.

Because the merge came first, the inventory panel was written against their version of the file. Doing it the other way round would have meant a conflict in the one file both of you were editing.

Habit worth keeping: merge `main` before you cut a branch, not after you finish one.

Part 10 — What comes next

Sprint 2 stands at **6 of 11** once this merges. The five left:

- Upgrade location map
- Locked / unlocked locations
- Show current player location
- Cloudinary setup
- Scenario media upload

The first three overlap the map work Person C touched — worth checking they are still yours before starting.

Checks

Check	Result
<code>npm test</code> (server, after merging Person C)	89 pass, 0 fail
<code>npm run lint</code> (client)	Clean, no warnings
<code>npm run build</code> (client)	Succeeded, 76 modules
Live: pick up City Map at the Forum	200, inventory <code>[city_map ×1]</code>
Live: pick up it again	409 <code>ALREADY_HAVE_ITEM</code>
Live: pick up bread, which lies at the bakery	409 <code>ITEM_NOT_HERE</code>
Live: pick up an item that does not exist	404 <code>ITEM_NOT_FOUND</code>
Live: use the City Map	409 <code>ITEM_NOT_USABLE</code>
Live: use bread, not carried	409 <code>ITEM_NOT_IN_INVENTORY</code>
Live: use with no item named	400 <code>VALIDATION_ERROR</code>
Test data	QA user and game deleted from the shared database

Not checked: how it looks. The Chrome extension dropped mid-session, so no screenshot was ever taken. The game page loaded and its API call returned 200, and lint and build pass — but card layout, the quantity badge and the red error styling have not been seen by a human or a camera. Worth one look in the browser before the demo.